

When QUIC Meets TCP: an Experimental Study

Yajun Yu^{*,†}, Mingwei Xu^{*,†}, Yuan Yang^{*,†}

^{*}Department of Computer Science and Technology, Tsinghua University, China

[†]Tsinghua National Laboratory for Information Science and Technology

Email: yu-yj16@mails.tsinghua.edu.cn; xmw@cernet.edu.cn; yyang@csnet1.cs.tsinghua.edu.cn;

Abstract—Recently, QUIC proposed by Google has drawn great attentions due to several attractive features which improve the page load time for Web applications. The features of QUIC lie across different protocol layers, and the benefits and the limitations introduced by each feature have not been understood sufficiently. In this paper, we focus on the transport aspect of QUIC, with the packet pacing mechanism for congestion control. We conduct an experimental study to evaluate the performance of QUIC as a general-purpose transport protocol, especially when QUIC is used concurrently with TCP, which is still the dominating transport protocol on the Internet. By doing so, we can exploit the potential of QUIC as a transport protocol in other application scenarios besides HTTP/2. It is also useful for the improvement of QUIC itself, because each object transmitted in one stream of a QUIC connection has to compete for resources with other transport protocols on the Internet. We do both testbed and real Internet experiments, on typical network environments such as 4G and WiFi. Our main findings indicate that, QUIC is less competitive than TCP in a network with little loss rate, large buffer, or large propagation delay; the 0-RTT connection establishment feature of QUIC has little advantage over TCP in practice; and the main benefit of QUIC lies in the multi-stream-based multiplexing mechanism.

I. INTRODUCTION

Most applications on the Internet leverage transport protocols for a variety of services, such as connection-based communication, reliable transmission, congestion control, etc. However, transport protocols has become the bottleneck for the services above after the rapid growth of the Internet and applications. In 2013, Google proposed QUIC (Quick UDP Internet Connections) [1], an advanced protocol used to improve the throughput and shorten the page load time for Web applications. Since QUIC was first born, great attentions have been drawn due to several attractive features. In particular, QUIC runs on the top of UDP, and establishes a connection in 0 or 1 round-trip-time (RTT). QUIC uses pluggable congestion control algorithms, including a packet pacing mechanism, whose basic idea is different from conventional congestion control methods that use a congestion window. Furthermore, QUIC allows multiple objects to be sent in separate streams, which are transmitted concurrently over the same connection. An error in one stream will not block other streams, so the head-of-line (HOL) blocking problem that degrades the performance of HTTP can be eliminated. The stream mechanism of QUIC also provides opportunities for further optimization, such as prioritization among streams and compression in the same stream.

Although QUIC is one protocol used as a whole, the functionalities lie across several layers in the ISO seven-layer refer-

ence model in Fig. 1. In particular, the UDP-based connection management and the congestion control algorithms belong to the transport layer, while the multiplexing mechanism using streams can be seen as in the session and presentation layers. We observe that the functionalities of QUIC in different layers are independent with each other. By decoupling such functionalities, we can have a better understanding of the benefits and the limitations introduced by each feature. In other words, it is not necessary to evaluate QUIC as a whole each time a new version is released. In particular, we are interested in the transport aspect of QUIC, as well as the packet pacing mechanism [2] for congestion control. It is important to understand the performance of QUIC as a general-purpose transport protocol, especially when QUIC is used concurrently with TCP, which is still the dominating transport protocol on the Internet. By doing so, we can exploit the potential of QUIC as a transport protocol in other application scenarios besides HTTP/2. It is also useful for the improvement of QUIC itself, because each object transmitted in one stream of a QUIC connection has to compete for resources with other transport protocols on the Internet.

In this paper, we conduct experiments on both a testbed and the real Internet. Unlike existing experimental studies using HTTP/2 as the application scenario in the testbed, we treat QUIC as a general-purpose transport protocol to transmit data, which is compared with TCP. We do a comprehensive experimental study. We first construct a network with a default configuration of bandwidth, propagation delay, buffer size, and loss rate. We evaluate the performance of QUIC and TCP on such a network which is set as baselines. Then, we observe the performance with changing parameters one by one. We consider not only the situation in which a QUIC or TCP connection is used separately, but also the situation where QUIC and TCP compete for a bottleneck link or shared buffer.

The key findings of this part of experiments can be summarized as follows. 1) QUIC has a throughput similar to TCP in a network with low loss rate, when the two protocols are used separately, but TCP outperforms QUIC when they compete. 2) QUIC has a throughput greater than TCP in a network with large loss rate or small buffer size, when they compete, but has an opposite effect when the protocols are used separately. 3) The performance of QUIC degrades on a network with large propagation delay, which can be much worse than TCP, in both scenarios. 4) Fairness can be achieved among different QUIC connections. And 5) With tradeoffs, the packet pacing mechanism can be regarded as efficient.

As for the experiments on real Internet, we also evaluate HTTP/2 over QUIC because there are no other application scenarios in practice currently. We change several parameters to evaluate QUIC, such as object number, object size, etc. We compare HTTP/2 over QUIC with HTTP/2 over TCP, and do the experiments in two typical network environments of 4G and WiFi. We find that the connection time of QUIC is similar to that of TCP or even worse, while the main benefit of QUIC lies in the transmission of a number of objects. Moreover, QUIC performs better in poor environment.

The rest of this paper is organized as follows. The development of HTTP and the details of QUIC's features are described in Section II. Section III reviews the related works. The experimental methods are illustrated in Section IV, followed by analyses of results in Section V. Finally, we conclude our works in Section VI and enumerate the future work in Section VII.

II. BACKGROUND

Since the invention of World Wide Web (WWW) in 1990, HTTP, a base protocol of WWW, has been developed gradually. In 1999, the first standard protocol of HTTP (i.e., HTTP/1.1 [3]) was published, and has stably run over 15 years. However, with the increasing variety of resources on the website and the rapid evolution of Internet, the flaws of HTTP/1.1 have been emerging, such as the HOL problem which will cause aggravating latency that is especially obvious in HTTP pipelining technology.

In 2009, Google proposed SPDY to enhance the performance and security of Web through compression, multiplexing, priority and the server push. In fact, SPDY just made progress on HTTP/1.1 in performance and was compatible with HTTP, so instead of replacing HTTP, in 2014 SPDY was integrated into HTTP which evolved to be HTTP/2 [4]. By August 2017, 15.8% of all the websites were already using HTTP/2¹. However, because the previous protocols are based on TCP which guarantees the packets arrive in order, they all face the HOL problem. Meanwhile, TCP's 3-RTT handshake establishing connection with TLS enlarges the establishment time while decreasing the latency is more effective than increasing the bandwidth to reduce the page load time (PLT)².

In order to solve the problems above, QUIC was proposed by Google in 2013 which is on top of UDP instead of TCP in transport layer. The architecture is illustrated in Fig. 1. For Google, they plan to put QUIC almost equivalent to an independent TCP connection, but with better SPDY-like stream-multiplexing support and much reduced latency.

An important property of QUIC is UDP-based multiplexing which can avoid the HOL problem shown in Fig. 2. If a packet lost in stream 1 over HTTP/2 based on TCP (Fig. 2(a)), the receiver cannot receive any packets which are sent behind the lost one. When there are some losses in stream 1 over QUIC (Fig. 2(b)), it will not block the transmission of other streams

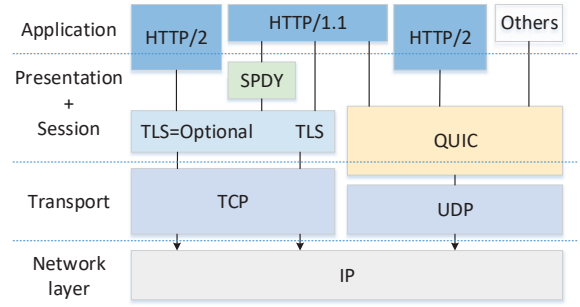


Fig. 1: The architecture of QUIC.

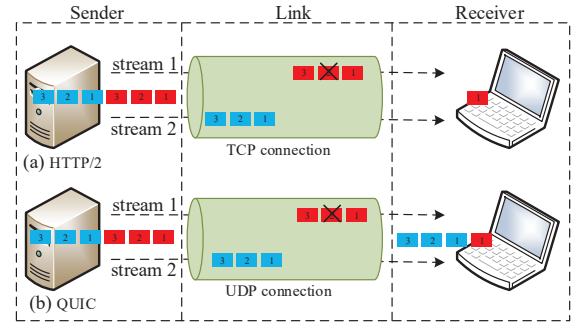


Fig. 2: Multiplexing comparison. The schematic about HOL problem over (a) HTTP/2, and (b) QUIC.

because there is no need to ensure an ordered reception. In theory, it can substantially reduce the latency. Another attractive feature of QUIC is 0-RTT connection establishment time which is also supposed to reduce the latency especially when there exists connection interruption. Moreover, there are many innovative mechanisms, such as the CID which identifies a connection and can achieve connection migration without interruption, and the packet pacing whose intention is to reduce the chance of loss and avoid congestion. Besides the traditional pluggable congestion control algorithm of adjusting the congestion window, there is a conserved mechanism, packet pacing, which is through injecting the waiting time between each UDP datagram, and it is enabled in QUIC by default. The scheme is shown in Fig. 3.

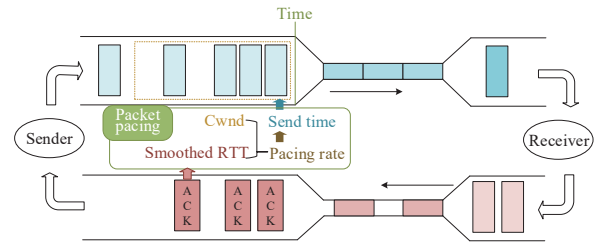


Fig. 3: The schematic of packet pacing.

¹<https://w3techs.com/technologies/details/ce-http2/all/all>

²<https://www.igvita.com/2012/07/19/latency-the-new-web-performance-bottleneck/>

III. RELATED WORK

Because QUIC is an innovative protocol proposed recently, there are relatively few studies. In [5] and [6], Google tests the influence of some mechanisms like FEC, then presents preliminary results and makes a simple performance test under simple scenarios in [7]. In recent years, there are also some performance-testing studies ([8]–[11]) and security-testing studies ([12]–[14]) by individual unit. Here we mainly focus on the performance test.

Somak Das [8] compares the PLT over HTTP/1.1, SPDY and QUIC in terms of different sizes of objects and network configurations in which bandwidth and delay vary. The author presents that QUIC reduces the load time of Web pages with small objects, and outperforms its predecessors by 10 - 60% over low-bandwidth and high-delay links. Later, Megyesi Péter et al. [9] verify the statement that which protocol performs best depends on the actual network conditions with an assembled measurement setup, as well as argue that QUIC is the fastest protocol by far on account of the 0-RTT connection time. In the same year, over more complex websites (Synthetic Pages/Complete Websites) and more various networks (Controlled Environment/Wired Networks/Cellular networks), Biswal Prasenjeet et al. [10] conclude that QUIC is faster when there are few small-sized objects, which is contrary to the more efficient multiplexing. It is speculated that increasing the number of embedded objects will reduce QUIC's performance due to inefficient browser queuing. In our paper, to visualize the effects of 0-RTT connection time, we investigate the connection establishment time in different network configurations. Apart from the PTL comparison over different network environments, object sizes and object numbers, we also research the influence of different loss and reordering rates. This part complements the predecessors' work.

Over QUIC, SPDY and HTTP/1.1, Carlucci Gaetano et al. [11] investigate the utilization while introducing losses and FEC (ON/OFF), and the fairness when QUIC/TCP flows coexist with different buffer sizes. The authors claim that QUIC is better than HTTP in loss-free channel and better than SPDY in a lossy channel, and FEC worsens the performance of QUIC and QUIC prevails over TCP in an under-buffered network. In our paper, considering the application of packet pacing mechanism in QUIC, we have a study on whether it is ineffective like FEC. Moreover, given that QUIC contains the main functions of transport layer, unlike work done by others, we focus on the transport aspect, and make a comparison on performance between TCP and QUIC in various network configurations, as well as different buffer sizes.

Most previous works compare HTTP/1.1, SPDY and QUIC, however, as mentioned before, HTTP/2 is prevailing on websites. Therefore, in our paper, we just compare HTTP/2+TLS+TCP and HTTP/2+QUIC+UDP (Fig. 1). From the transport aspect, we are interested in the performance of QUIC compared to TCP. Combining the application with HTTP/2, we investigate the improvements of QUIC in practice compared to the traditional TCP-based HTTP/2.

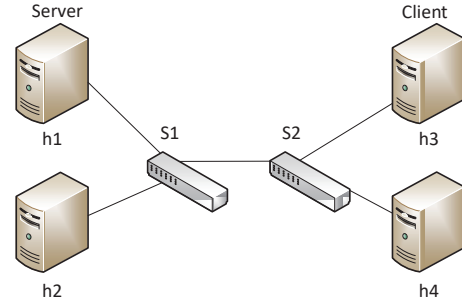


Fig. 4: The dumbbell topology.

IV. EXPERIMENTAL METHODS

According to the open-source experimental QUIC, our experiments contain three parts: the performance as a part of transport layer, the effects of packet pacing mechanism and the improvements with HTTP/2. The first aims to find whether the QUIC is fair with TCP through observing the throughput in different scenarios on our testbed, the second one investigates the effects of packet pacing mechanism to verify its efficiency, and by recording the load time of Web pages under different network conditions, the last researches the improvements of QUIC when it applies with HTTP/2 on real Internet. This section will explain the process of experiments in detail.

A. The performance as a part of transport layer

To analyze the performance of QUIC and TCP, with *Mininet* emulation platform on Ubuntu14.04, we build a testbed on which we apply a typical dumbbell topology shown in Fig. 4. The link between s1 and s2 is the bottleneck link that we control to configure the bandwidth, the delay and the loss. And the relevant configurations are listed in Table I(a) when the bandwidth and delay of the rest links (e.g., from h3 to s2) are set with 1Mbps and 20ms respectively and the buffer size is set by default which is 1000 here. From a client to a server, in the competitive scenario, h3 sends a random stream based on TCP to h1, and h4, a QUIC client, sends a random stream based on QUIC to h2, a QUIC server, both of which are compiled based on *libquic* library³. Here the congestion control algorithm is *Cubic*, and in each experiment which lasts 30s, we record the received bytes on each server each second. Apart from configuring different networks, we also set various buffer sizes of s2 in Table I(b) with the same delay (=20ms) and loss (=0%) configurations to research the influence of buffer size. Note that the loss rate refers to random packet loss rate of link here, the value of bandwidth we set is to ensure that the sending rate is not the bottleneck, and buffer size is about the number of packets.

In this experiment, by computing the link utilization that we define in equation 1, we analyze the fairness of bandwidth allocation with different network configurations when flows traverse the bottleneck link. The *tot_time* is the time of each experiment lasts (here is 30s), the *GoodPut* is the rate of receiving bytes, and the *BandWidth* is the minimum

³<https://github.com/devsisters/libquic>

TABLE I: The network configurations on testbed

(a)	Bandwidth (Mbps)	1, 2
	Delay (ms)	20, 100
	Loss (%)	0, 2, 10
(b)	Bandwidth (Mbps)	1, 2
	Buffer size	2, 15, 100

bandwidth of path, that is the minimum value between the bandwidth of bottleneck link and the one of rest links, we define the *Utilization* is the mean of the amount of real received bytes divided by the amount of expected received bytes. All results will be introduced in next section.

$$Utilization = \frac{1}{tot_time} * \sum_{t=1}^{tot_time} \frac{GoodPut * t}{BandWidth * t} \quad (1)$$

B. The effects of packet pacing mechanism

In this mechanism, there are some bandwidth estimators that will estimate the maximum available bandwidth and according to the estimate set an appropriate packet pacing to transmit the data packets. It is believed that in the packet flow, pacing reduces fluctuations, and thereby eases the congestion based losses (i.e., packet drop in switches due to overflows). [15] However, to meet the pacing rate, the injection of wait time will also induce some latency and reduce the throughput. In this subsection, we investigate that whether the mechanism is similar to FEC proved to be inefficient.

This experiment is based on the same setup of Table I(b) in Section IV-A. The differences are that we focus on the number of lost packets with enabling and disabling packet pacing. And meanwhile, we compare the throughput under the two different cases.

C. The application with HTTP/2

On real Internet, we evaluate HTTP/2 over QUIC and TCP, and build a system shown in Fig. 5. As for server, we choose Caddy, which supports HTTP/2, moreover, it is a QUIC-supported Web server which can eliminate the influence of server differentiation. We have pages with different sizes and various numbers to measure the connection time (CT) and PLT. For the availability and the reproducibility of experiments, these Web pages are all artificial and the setup of parameters is shown in Table II in which the size of page consisting of 14 objects is 1.5MB. Furthermore, to prove the universality of the results of this experiment, we also measure the PLT of a real website (YouTube.com) which has applied QUIC.



Fig. 5: The structure over Internet.

TABLE II: The parameters setting on real Internet

Object size	2KB, 100KB, 1.5MB
Object number	1, 14
Net configuration	Regular 4G (20ms, 4.0Mbps), WiFi (2ms, 30Mbps)
	Loss (%): 0, 1, 5
	Reorder (%): 0, 10, 30

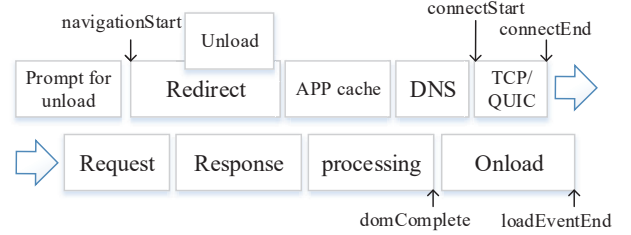


Fig. 6: The processing model of loading pages.

The client is Chrome browser, in which we can enable or disable QUIC manually (*chrome : //flags/*) or through command line. Under Windows, we control the network through the *WANem* tool which is deployed on a virtual machine as a switch, and the network configurations also can be found in Table II. The reason why we select these types of network configurations is that the configurations are close to the common realistic WIFI and 4G environments.

All network requests are seen as resources and they have different life cycles, the processes can be seen in Fig. 6, we obtain a large number of detailed information through the Performance Web API which is an interface providing Web applications with timing-related information. To reduce the influence of cache, we disable the HTTP cache function. And in the processing model, at the time of *domComplete*, the current document is ready, it can be regarded as the preparation stage and contains the connection establishment. The *Onload* stage fires load event which occupies most of time and really depends on the size of page. Therefore, we analyze the load time including CT and PLT computed by Equation 2 and 3 and repeat each experiment 60 times.

$$CT = connectEnd - connectStart \quad (2)$$

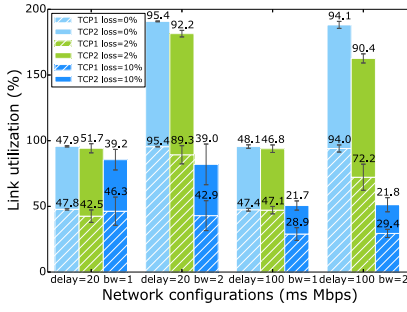
$$PLT = loadEventEnd - navigationStart \quad (3)$$

V. RESULTS AND DISCUSSION

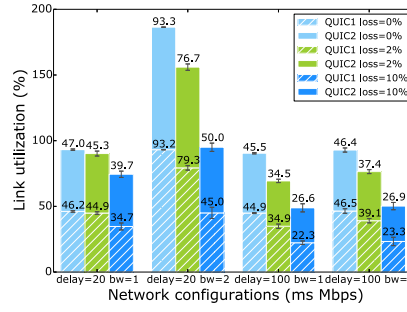
This section contains detailed experimental results based on section IV. Like the latest section, we analyze the results from three parts in details.

A. The performance as a part of transport layer

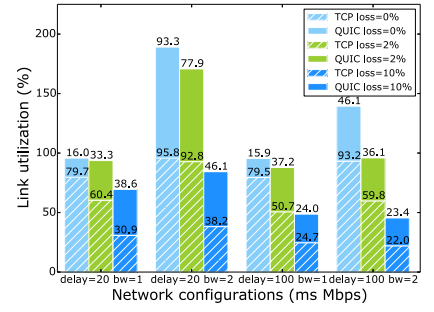
1) *When the protocols are used individually with different network configurations:* To survey the competition between TCP and QUIC, firstly we have done two experiments, one only contains 2 TCP flows (Fig. 7(a)) while the other transfers 2 QUIC flows (Fig. 7(b)). We can see that in each one and over each network configuration, the same kind flows share the bandwidth of bottleneck fairly in general which



(a) 2 TCP

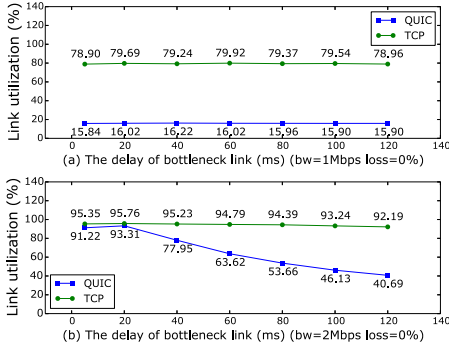


(b) 2 QUIC

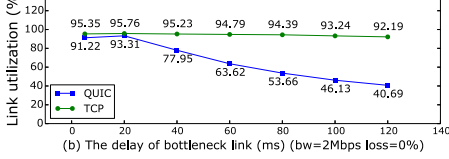


(c) 1 TCP+1 QUIC

Fig. 7: The link utilization with different network configurations and different types of flows.



(a) The delay of bottleneck link (ms) (bw=1Mbps loss=0%)



(b) The delay of bottleneck link (ms) (bw=2Mbps loss=0%)

Fig. 8: The link utilization with different values of delay while TCP/QUIC flows coexist (i.e., Fig. 7(c)).

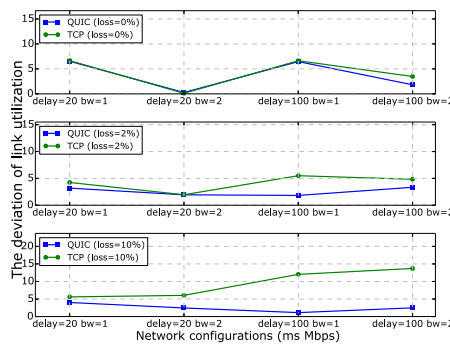


Fig. 9: The deviation of utilization with different network configurations while TCP/QUIC flows coexist (i.e., Fig. 7(c)).

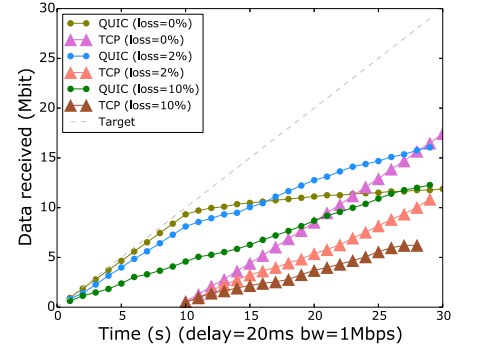


Fig. 10: A experiment where QUIC flow followed by TCP flow after 10s.

is reasonable intuitively. Meanwhile, through observing the deviation primitively, it is not difficult to find that QUIC is much more stably than TCP. In Fig. 7(a), by comparing the first/third bar and the second/fourth bar where the bandwidth is same while the delay is different, we find that the correlation between link utilization and delay is not so strong for TCP flows. However, while bandwidth is 2Mbps and delay is 100ms, over QUIC (Fig. 7(b)), the link utilization is reduced by half than while bandwidth is 2Mbps and delay is 20ms. In a word, if there is not bottleneck link, the link utilization is related to delay for QUIC. There are similar phenomena while two kinds flows coexist over two protocols (Fig. 7(c)).

2) *When two types of flows coexist with different network configurations:* We extend the third experiment to make more thorough investigations. With more fine-grained delay seen in Fig. 8 where the loss is 0%, there are two major discoveries. When bandwidth is 1Mbps (i.e., the resources are scarcity) shown in Fig. 8(a), the resources TCP flows occupied is 5 times to that QUIC flows obtained which is unfair dramatically, and is irrelevant to delay. The utilization of QUIC is only 16% on average, to some extent, it is the packet pacing mechanism in QUIC that aggravates the result because the estimator will estimate there is not enough available bandwidth and increase the wait time. If there is no competition for resources (Fig. 8(b)), after the delay exceeds

a threshold, with the increasing of delay, the link utilization of QUIC flows is decreasing, while the TCP flows keep stable. We speculate that with a larger delay, the period of adjusting endpoint's flow control window is longer (i.e., the cycle of sending *WINDOW_UPDATE* frame. [1]) which causes the degradation of performance.

In addition, according to the statistics of deviations in Fig. 9, while two kinds flows coexist, the standard deviation of TCP is larger than QUIC prevalently, especially there are some losses, which is same as the preliminary observation while data transfer over one of the protocols individually. These mean that QUIC is much more stably and the reason should be attributed to packet pacing which makes sure the send rate is not too aggressive, while TCP sends packets as soon as possible.

When it comes to a lossy network and the TCP/QUIC flows coexist, the inferior position of QUIC has some changes. When the loss of network is aggravated from 0% to 10% shown in Fig. 7(c), the bandwidth allocation of QUIC is greater than TCP almost 12%, which is the effect of conserved mechanism.

In order to show the unfairness between TCP and QUIC more visually, we inject the TCP flow 10s later after the QUIC flow starts shown in Fig. 10. The gray dotted line represents the except received bytes whose gradient equals to the bandwidth of path. At the beginning, in 0-10s, the received rate of QUIC approaches to expected value, afterwards with the join of TCP,

TABLE III: The proportion between buffer size and BDP

Buffer size	2	15	100
# BDP	0.13	1	6.67

all of the received rates have the downtrend, especially under loss-free network in which the rate of QUIC is close to zero, it even can not seize any bandwidth while the rate of TCP equals to the target, which accords with the result that the average of link utilization of QUIC is only 16% in Fig. 7(c). Only under a high-loss network (e.g., loss=10%), can QUIC compete with TCP.

3) *The performance with different buffer sizes:* Finally, we focus on the different buffer sizes shown in Fig. 11 under identical scenarios as above. When bandwidth is set to 2Mbps which means there will not have a queue in switch, obviously, except the *maxQLens* is too small, the bandwidth allocation of TCP equals to that of QUIC approximately. If bandwidth equals to 1Mbps, the different buffer sizes correspond to the different proportions of BDP seen in Table III which represents the packet loss probability due to overflow.

In Fig. 11(a) and Fig. 11(b), the protocols are used individually, they can share the resources fairly, but with small buffer size, which means heavy congestion, TCP outperforms QUIC, this is because QUIC has a more conserved congestion control system. However, when they compete, QUIC prevails over TCP in under-buffered networks seen in Fig. 11(c).

We extract the results where bandwidth is 1Mbps and delay is 20 ms (i.e., baseline) in Table IV and Table V respectively, and find that with larger loss rate and smaller buffer size, the performance of QUIC is better. While the loss is 0, the ratio is only 0.2 which implies the heavy unfairness. Considering the loss rate of traditional network is low and the buffer size is limited, so the performance of QUIC really depends on the number and the type of flows.

TABLE IV: The utilization ratio of QUIC to TCP with different loss rates

Loss rate	0	5	10
Utilization ratio	0.2	0.55	1.25

TABLE V: The utilization ratio of QUIC to TCP with different buffer sizes

Buffer size	100	15	2
Utilization ratio	0.22	0.87	5

B. The effects of packet pacing mechanism

First of all, we observe the effects of the mechanism on packet losses in a competitive scenario. As shown in Fig. 12(a) where bandwidth is 1Mbps and in Fig. 12(b) where bandwidth is 2Mbps, while the packet pacing mechanism is enabled, the number of losses is redeuced by half approximately, compared to the case of disabling the mechanism.

In Fig. 12(c), we can find that under limited buffer size, the packet pacing mechanism improves the throughput over 50%. As expected, under good environments (i.e., buffer size is equal or greater than BDP), the throughput of QUIC is reduced

TABLE VI: The mean of connection time (HTTP/QUIC)

Time (ms) \ N.C.	30M 2ms	4M 20ms
Number		
1	85.7/93.1	135.4/137.2
14	84.6/98.6	137.1/132.1

about within 20%. The results are same in other scenarios. Therefore, with tradeoffs, the packet pacing mechanism is an efficient mechanism, especially in under-buffered networks.

C. The application with HTTP/2

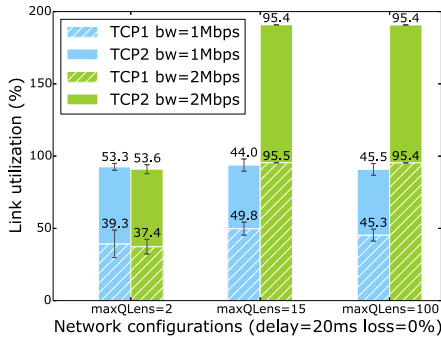
In this part, given the space limitation and to investigate the most significant improvements of QUIC, we display main results shown in Fig. 13-19. Note that in this section we use HTTP/2 represents HTTP/2 over TCP while QUIC represents HTTP/2 over QUIC.

1) *The PLT with different network environments:* In Fig. 13, we research the influence of network environments with related object number. From the two sub-graphs, we can find that under WIFI network, HTTP/2 is superior to QUIC and much more stably. we guess it is because of the conservative packet pacing which limits the rate of transfer, then causes the larger load time range. Under 4G network which represents the poor environment, QUIC is faster than HTTP/2 around 11% at most which is because of packet pacing reducing the chance of congestion and multiplexing without HOL problem.

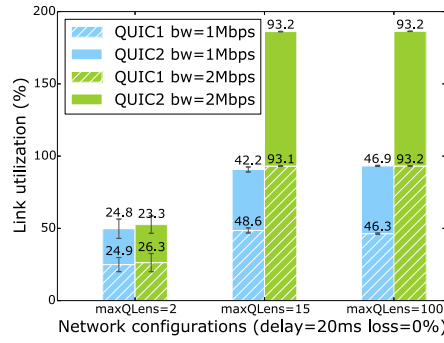
2) *The CT with different network environments:* It is known that one main feature of QUIC is 0-RTT, therefore we compare the connection time to observe the improvements it effects. However, it is not so remarkable as expect seen in Fig. 14. Comparing the mean of connection time (Table VI), we find only under 4G network, QUIC gets upsides with HTTP/2. Under WIFI network, QUIC is worse than HTTP/2 by 8.6% at least. During the experiment, we observe that when the Web enables QUIC, not all resources are transferred over QUIC, and the two kinds connections are concurrent. So in practice, this fails to affect fully, even works much worse.

3) *The improvements with different object sizes and numbers:* Seen in Fig. 15, if there are few object numbers, the correlation between the page size and the improvements of QUIC is not so obvious, even in poor environment. But the larger object number is, the greater the improvement of QUIC makes, shown in Fig. 16 where the improvement of HTTP/2 is 18.4% while that of QUIC is 26.6%. Thus, it is speculated that the main benefit of QUIC lies in the transmission of the object number, rather than the object size.

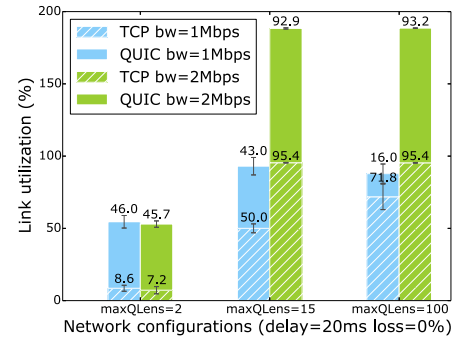
4) *The improvements with different loss and reorder rates:* As for the investigations of different loss and reorder rates of network, to eliminate the preponderance of QUIC in poor network, we discuss the improvements of QUIC in WIFI network. In Fig. 17, QUIC changes the inferiority in a lossy network compared to in loss-free network. What's more, the bigger the possibility of loss is, the better QUIC is, and the improvement ranges from 3.0% to 9.5%, which can be owed to the larger NACKs range which can feedback more



(a) 2 TCP

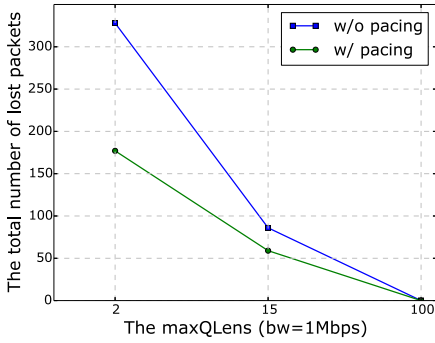


(b) 2 QUIC

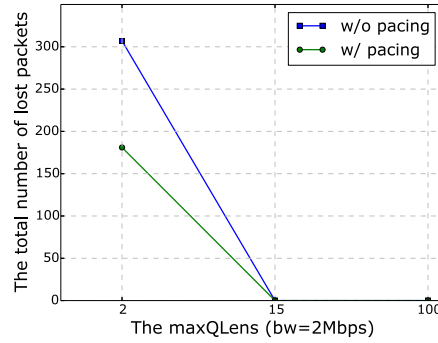


(c) 1 TCP+1 QUIC

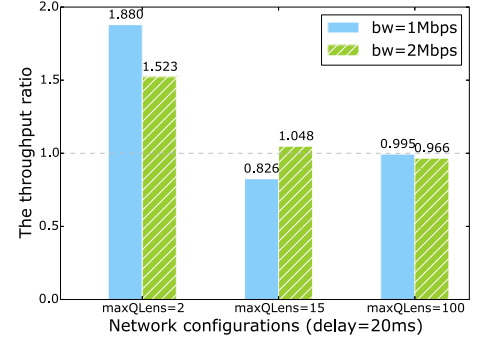
Fig. 11: The link utilization with different buffer sizes and different types of flows.



(a) The comparison on the number of losses in bw=1Mbps

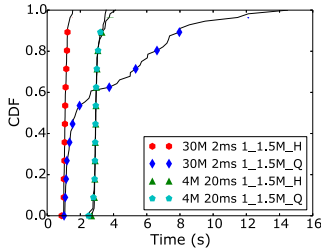


(b) The comparison on the number of losses in bw=2Mbps

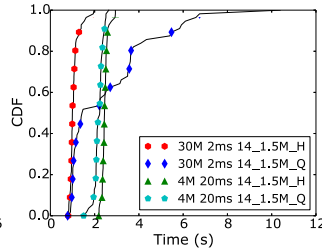


(c) The throughput ratio of enabling to disabling pacing

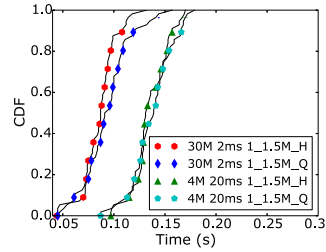
Fig. 12: The effects of packet pacing.



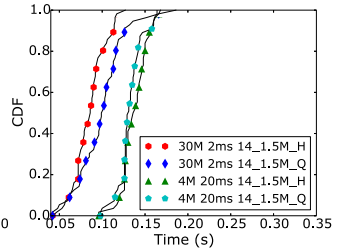
(a) Object number=1



(b) Object number=14



(a) Object number=1



(b) Object number=14

Fig. 13: The PLT with different network configurations and different object numbers. (The labels mean “bw delay number_size_protocol”, the same below)

Fig. 14: The CT with different network configurations and different object numbers.

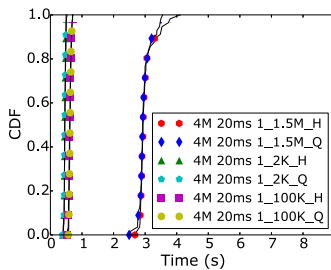


Fig. 15: The PLT with different object sizes.

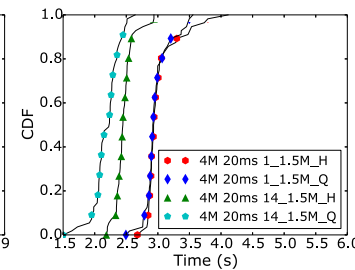


Fig. 16: The PLT with different object numbers.

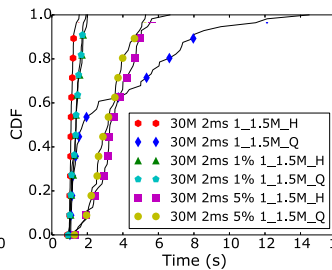


Fig. 17: The PLT with different loss rates.

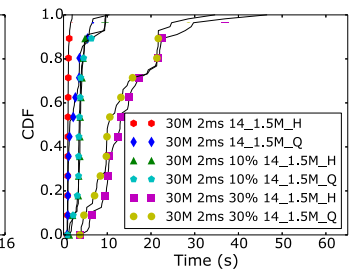


Fig. 18: The PLT with different reorder rates.

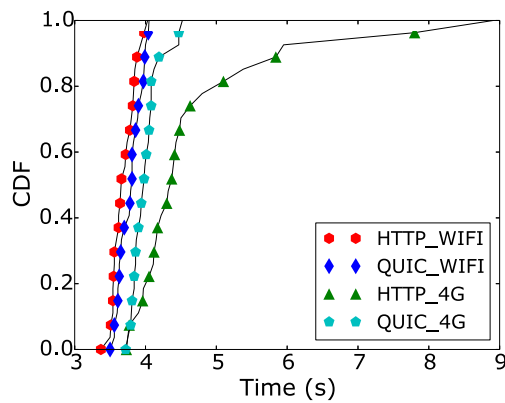


Fig. 19: The PLT of loading YouTube.com.

information and without the effect of HOL problem. In out-of-order network, there is similar finding shown in Fig. 18.

5) *A preliminary verification:* In Fig. 19, we measure the PLT of loading YouTube.com in which there are 1.5M bytes transferred totally each time, and obtain the results that QUIC is superior to HTTP/2 in poor network, but worse in good network, which are in accord with the above experiments.

VI. CONCLUSION

QUIC is a UDP-based protocol, and also realizes reliability and security, whose aim is to reduce the latency for Web pages recently. Its main feature is 0-RTT connection establishment time, what's more, it has more alternative options such as packet pacing mechanism and all benefits of SPDY such as multiplexing. Experiments have proved that the performances really depend on the network conditions.

As a variation of transport protocol, we evaluate the performance and fairness between QUIC and TCP. When they compete for resources, QUIC has a throughput greater than TCP in a network with large loss rate or small buffer size, that is because QUIC need not to ensure packets arrive in order, and own a conservative congestion control mechanism. However, QUIC is far behind TCP in loss-free or unlimited-buffer network, and is affected by delay. Fairness can only be achieved among different QUIC connections.

To some extent, packet pacing lowers the congestion related packet loss and makes QUIC much more stably which limits the performance in low loss network at the same time. After tradeoffs, it is proved to be an efficient mechanism.

As for the application with HTTP/2, there are some discoveries. Different from intuitional thought, the 0-RTT feature has not made great improvements, because two kinds connections are concurrent if QUIC is enabled in practice. Due to the exist of multiplexing in both two protocols, the main advantage of QUIC is embodied in no HOL problem. In general, QUIC is superior in poor network while is limited in great network with the packet pacing mechanism. And QUIC makes more improvements with larger object number.

There is no doubt that QUIC is a promising protocol which breaks the routine, although it needs more all-sided considerations to deploy widely. QUIC behaves well in poor

environments like 4G network. Therefore, maybe it can be an alternative to implement for Android which has high mobile traffic. Different from traditional transport protocol, QUIC can be faster iterated and deployed, we can also regard it as a platform to debug some new ideas, not just a protocol.

VII. FUTURE WORK

QUIC is a booming protocol, there are many works we can do. According to our conclusions, QUIC has much better performances in poor environment. Considering the features of space network's link (e.g., high latency, high bit error rate, connection interruption, etc.), we plan to design a suitable transport protocol for space network inspired by QUIC.

Recently, QuicTransfer⁴ is an application based on QUIC, which is a cloud-based file transfer service. We are considering the application on any other types of resources such as real streaming, or with any other protocols, not just HTTP.

As we know, safety is an eternal topic. One of the highlights of QUIC is 0-RTT, maybe we can take advantage of it to mitigate the low-rate attack [16] in TCP, however, on the contrary if adversaries make QUIC fall back to TCP/TLS or an inconsistent view of handshake stage between client and server, these attacks can lead to more latency and inconsistent state. So this is worth studying.

REFERENCES

- [1] R. Hamilton, J. Iyengar, I. Swett, and A. Wilk, "QUIC: A UDP-based secure and reliable transport for HTTP/2," *IETF draft-tsvwg-quic-protocol-02*, 2016.
- [2] D. Wei, P. Cao, and S. Low, "Tcp pacing revisited," in *IEEE INFOCOM*, 2006.
- [3] R. Fielding, J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, and T. Berners-Lee, "Hypertext transfer protocol-HTTP/1.1," *IETF RFC 2616*, 1999.
- [4] M. Belshe, M. Thomson, and R. Peon, "Hypertext transfer protocol version 2 (HTTP/2)," *IETF RFC 7540*, 2015.
- [5] R. Jim, "QUIC: Multiplexed stream transport over UDP," <http://www.ietf.org/proceedings/88/slides/slides-88-tsvarea-10.pdf>, 2013.
- [6] "BarBoF: Congestion control and loss recovery," Tech. Rep., 2015.
- [7] A. Gizis, "Taking Google's QUIC for a test drive," <http://www.connectify.me/blog/taking-google-quic-for-a-test-drive/>, 2013.
- [8] S. R. Das, "Evaluation of QUIC on web page performance," Ph.D. dissertation, Massachusetts Institute of Technology, 2014.
- [9] P. Megyesi, Z. Krämer, and S. Molnár, "How quick is QUIC?" in *IEEE ICC*, 2016, pp. 1–6.
- [10] P. Biswal and O. Gnawali, "Does QUIC make the Web faster?" in *IEEE GLOBECOM*, 2016, pp. 1–6.
- [11] G. Carlucci, L. De Cicco, and S. Mascolo, "HTTP over UDP: an experimental investigation of QUIC," in *ACM SAC*, 2015, pp. 609–614.
- [12] R. Lychev, S. Jero, A. Boldyreva, and C. Nita-Rotaru, "How secure and quick is QUIC? provable security and performance analyses," in *IEEE SP*, 2015, pp. 214–231.
- [13] T. Jager, J. Schwenk, and J. Somorovsky, "On the security of TLS 1.3 and QUIC against weaknesses in PKCS# 1 v1. 5 encryption," in *ACM CCS*, 2015, pp. 1185–1196.
- [14] M. Fischlin and F. Günther, "Multi-stage key exchange and the case of Google's QUIC protocol," in *ACM CCS*, 2014, pp. 1193–1204.
- [15] J. Roskind, "QUIC: Design document and specification rational," Tech. Rep., 2015.
- [16] A. Kuzmanovic and E. W. Knightly, "Low-rate TCP-targeted denial of service attacks: the shrew vs. the mice and elephants," in *ACM SIGCOMM*, 2003, pp. 75–86.

⁴<https://quictransfer.com/>